

5

Setting Up DbContext and Controllers

Routing and controllers are responsible for receiving, validating, and processing incoming HTTP requests. During the processing, the controllers may or may not persist and read records in the database through `DbContext`. Learning how a controller and `DbContext` work together is essential in building web application projects that work as expected.

At the end of this chapter, you will be able to understand the following topics:

- Writing entities and enums
- Setting up a database, EF Core, and `DbContext`
- Writing controllers and routes
- Testing controllers with Swagger UI

Technical requirements

Here is what you need to complete this chapter:

- Visual Studio 2019, Visual Studio for Mac, or Rider
- The `dotnet-ef` tool
- SQLite Browser or SQLiteStudio

Here is the starter repository for this chapter, which is also the finished source code of *Chapter 4, Applying Clean Architecture ASP.NET Core Solution*: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter05>.

Writing entities and enums

Here are quick definitions of entity and enum before we start writing them in the domain project. What is an entity? An **entity** is a representation of a domain object model in an application. A database translates an entity into a row in a database table. Similarly, the properties of an entity are the columns in a database table.

On the other hand, an **enum** is a type of class representing a set of constants or read-only variables.

We have looked at a quick definition of an entity and an enum; now, let's see them in action.

Creating entities and enums for the Travel Tour application

In your solution application, go to the `Travel.Domain` project and create two directories:

- `Entities`
- `Enums`

The Entities directory will be for all the Travel Tour application entities, while the Enums directory is for all the Travel Tour application enums:

1. In the Entities folder, create a C# file, `TourPackage.cs`, and write this code:

```
namespace Travel.Domain.Entities
{
    public class TourPackage
    {
        public int Id { get; set; }
        public int ListId { get; set; }
        public string Name { get; set; }
        public string WhatToExpect { get; set; }
        public string MapLocation { get; set; }
        public float Price { get; set; }
        public int Duration { get; set; }
        public bool InstantConfirmation { get; set; }
    }
}
```

The class is a `TourPackage` entity with a namespace of `Travel.Domain.Entities`. The `TourPackage` entity represents the package tour data in the database, which has `Id`, `ListId`, `Name`, `WhatToExpect`, `MapLocation`, `Price`, `Duration`, and `InstantConfirmation`. It also needs an enum for `Currency` and an object relationship for `List`. Now let's create these two classes.

2. In your Enums folder, create `Currency.cs` and write the following code:

```
namespace Travel.Domain.Enums
{
    public enum Currency
    {
        PHP,
        USD,
        JPY,
        EUR,
        NOK
    }
}
```

The Currency enum file has a namespace of `Travel.Domain.Enums` and enumeration of PHP, USD, JPY, EUR, and NOK.

3. Now switch back to the `TourPackage` class to add the Currency enum like so:

```
using Travel.Domain.Enums;

namespace Travel.Domain.Entities
{
    public class TourPackage
    {
        ...
        public int Duration { get; set; }
        public bool InstantConfirmation { get; set; }
        public Currency Currency { get; set; }
    }
}
```

Bring in the `Travel.Domain.Enums` namespace so you can use the Enum class you just created by using `Travel.Domain.Enums`. Now add the Currency property after the `InstantConfirmation` property.

4. Next, let's create a `TourList.cs` C# file inside the `Entities` folder and write the following code:

```
using System.Collections.Generic;

namespace Travel.Domain.Entities
{
    public class TourList
    {
        public TourList()
        {
            Tours = new List<TourPackage>();
        }

        public IList<TourPackage> Tours { get; set; }
        public int Id { get; set; }
        public string City { get; set; }
        public string Country { get; set; }
    }
}
```

```
        public string About { get; set; }  
    }  
}
```

Let's bring in `System.Collections.Generic` and define the `TourList` entity. The `TourList` entity has a constructor that initializes the `Tours` property with a new `List` type of `TourPackage`, `new List<TourPackage>()`, which sets a one-to-many relationship. And the rest of the properties are `Id`, `City`, `Country`, and `About`.

5. Now go back to `TourPackage.cs` and add a `List` property of type `TourList` below the `Currency` property:

```
using Travel.Domain.Enums;  
  
namespace Travel.Domain.Entities  
{  
    public class TourPackage  
    {  
        ...  
        public bool InstantConfirmation { get; set; }  
        public Currency Currency { get; set; }  
        public TourList List { get; set; }  
    }  
}
```

The `List` type `TourList` signifies a one-to-one relationship, also known as an object-to-object relationship.

Writing the entities and enums for the domain project ends here. Now let's move on to the next section to set up **Entity Framework Core (EF Core)**, `DbContext`, and the database we will use in the application.

Setting up a database, EF Core, and DbContext

You often require a persistence framework, a middleware that helps developers store data in a database if there's a requirement to access a database in our application. Using a persistence framework, you can easily use your entities to query or save objects to a database. Now let's look at what EF Core and `DbContext` are.

EF Core

In ASP.NET Core, you can build a persistence framework from scratch, but you don't have to because it is time-consuming and expensive. Why? Writing many stored procedures is hard to maintain; reading data through ADO.NET objects then mapping them to your tables in your application is painful.

So, here comes Entity Framework to the rescue. **Entity Framework** is a persistence framework that does all the plumbing for you. Typically, you don't need to write any store procedures or map tables to your entities.

So what is EF Core? **EF Core** is a cross-platform **object-relational mapper (O/RM)**. EF Core itself is a version of Entity Framework, which is a persistence framework that uses the O/RM behind the scene to store and retrieve objects.

Okay, so that's an overview of EF Core; let's head to DbContext.

DbContext

EF Core provides a class called DbContext, which is an interface to our database. DbContext can have one or more DbSet that represents tables in our database. We use **Language Integrated Query (LINQ)** to query DbSet entities, and then EF Core will translate our LINQ queries to SQL queries at runtime.

DbContext opens a connection to your database, reads and maps data to objects, and adds them to DbSet of DbContext.

As we save, update, or delete objects in DbSet, EF Core keeps track of the changes. When we ask to persist the changes, EF Core automatically generates SQL statements and executes them in the database.

So that's a quick overview of DbContext; now it's time to write DbContext and some DbSet entities in the application.

Setup

This setup is going to be a simple EF Core and DbContext setup. However, we will refactor and improve this along the way:

1. First things first, install the `dotnet-ef` tool globally in your Terminal or CMD:

```
dotnet tool install --global dotnet-ef
```

The `dotnet-ef` tool is an EF Core tool for the .NET CLI. You can go to the link nuget.org/packages/dotnet-ef to see the latest version of the `dotnet-ef` tool.

We also need to install two NuGet packages. We will be using `dotnet cli` because it runs on any operating system, but if you prefer to use **Manage NuGet Packages**, a NuGet packages installer UI in your IDE, you can do so.

2. Now go to the `Travel.WebApi` project and run this command:

```
dotnet add package Microsoft.EntityFrameworkCore.Design
```

`Microsoft.EntityFrameworkCore.Design` is required in database migration using EF Core at runtime.

3. Next, go to the `Travel.Data` project and run this command:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

`Microsoft.EntityFrameworkCore.Sqlite` is the SQLite database provider of EF Core.

4. Now, still inside the `Data` project, let's write `DbContext` we will need here. Create a folder named `Contexts` inside the `Travel.Data` project. Inside the folder, create a `TravelDbContext.cs` file and write the following code:

```
using Microsoft.EntityFrameworkCore;
using Travel.Domain.Entities;

namespace Travel.Data.Contexts
{
    public class TravelDbContext : DbContext
    {
        public TravelDbContext
            (DbContextOptions<TravelDbContext> options)
            : base(options)
        {
        }

        public DbSet<TourList> TourLists { get; set; }
        public DbSet<TourPackage> TourPackages { get;
```

```
        set; }  
    }  
}
```

The preceding code imports the `Microsoft.EntityFrameworkCore` package as well as `Travel.Domain.Entities`. We named the class `TravelDbContext` and derived it from `DbContext` of EF Core.

The constructor of `TravelDbContext` has a parameter option of the `DbContextOptions` type of `TravelDbContext`, which goes to the base keyword.

You will see the two `DbSet` entities – `TourLists` and `TourPackages` – below the constructor.

5. Now go to the `Startup.cs` file of the `Travel.WebApi` project and bring in these two namespaces:

```
using Microsoft.EntityFrameworkCore;  
using Travel.Data.Contexts;
```

6. Write the following code inside the `ConfigureServices` method:

```
services.AddDbContext<TravelDbContext>(options =>  
    options  
        .UseSqlite("Data  
            Source=TravelTourDatabase.sqlite3"));
```

The code registers `TravelDbContext` as a service in `IServiceCollection` while `UseSqlite` configures the context to connect to the SQLite database.

The `ConfigureServices` method should look like this:

```
public void ConfigureServices(IServiceCollection  
    services)  
{  
    services.AddDbContext<TravelDbContext>(options  
        => options  
            .UseSqlite("Data  
                Source=TravelTourDatabase.sqlite3"));
```



```
services.AddControllers();  
services.AddSwaggerGen(c =>  
{  
    c.SwaggerDoc("v1", new OpenApiInfo { Title =  
        "Travel.WebApi", Version = "v1" });  
});  
}
```

7. Now let's go back to the `Travel.Data` project for database migration.

Note

All the commands are one-liner commands. The width of a book's page adds another line to any commands that are too long.

8. After navigating to the `Travel.Data` project, you are now ready to run your first migration:

```
dotnet ef migrations add InitialCreate --startup-project  
../../presentation/Travel.WebApi/
```

EF Core will create a folder named `Migrations` in your project with generated C# files. These files are the migration builder named `InitialCreate` and a snapshot of your database's current schema.

9. Next, create a database and schema from the migration files:

```
dotnet ef database update --startup-project ../../  
presentation/Travel.WebApi/
```

This command creates an SQLite database file, `TravelTourDatabase.sqlite3`, in the Web API project.

10. To see the successful migration that you did, use `SQLiteStudio` or `SQLite Browser`.

11. Add the `TravelTourDatabase` SQLite database file in `SQLiteStudio` or `SQLite Browser` to view the tables:

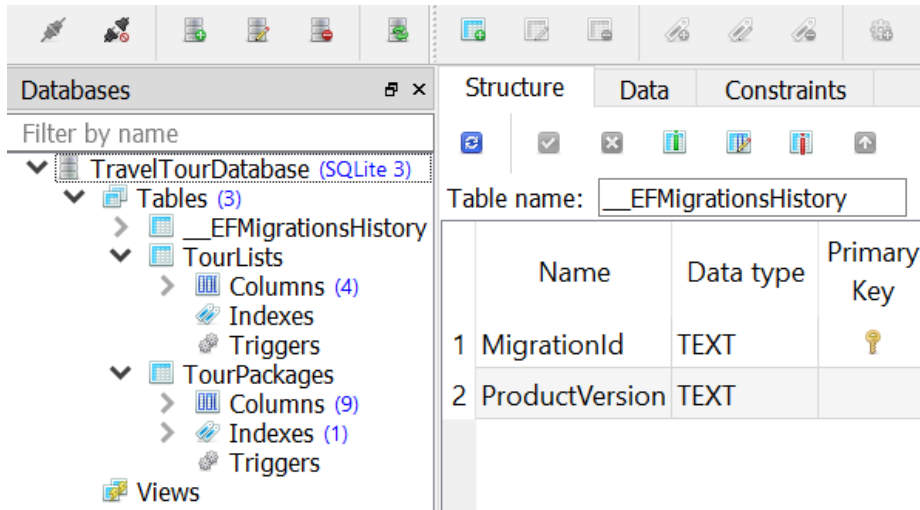


Figure 5.1 – SQLiteStudio

The preceding screenshot of `SQLiteStudio` shows `TravelTourDatabase`, while the following screenshot of `SQLite Browser` shows `TravelTourDatabase`:

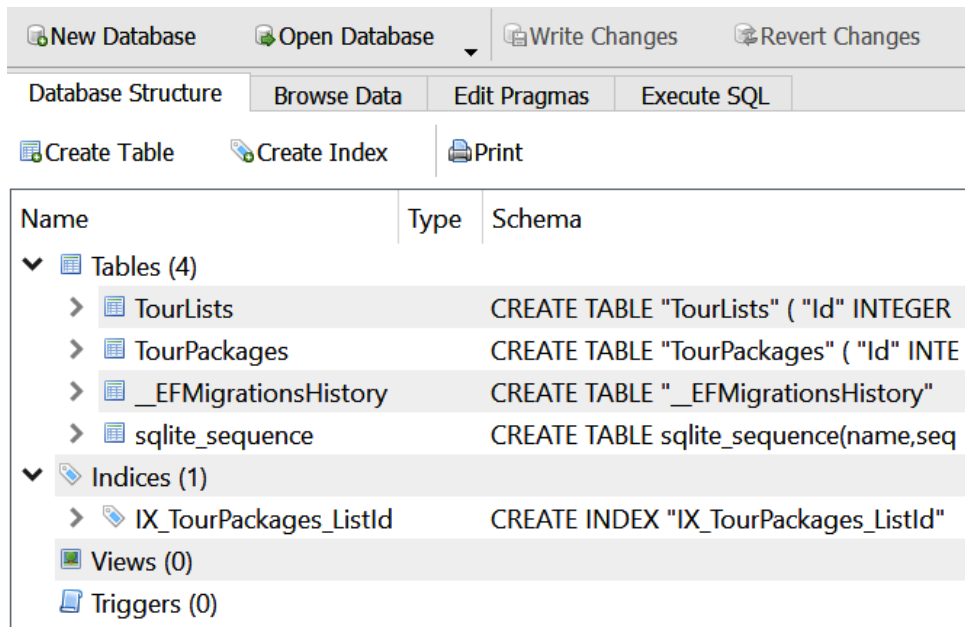


Figure 5.2 – SQLite Browser

You've just learned what entities, Entity Framework, EF Core, and `DbContext` are. You also learned where to get them and how to set them up. Now we have a database, let's move on to the next section, *Writing controller and routes*.

Writing controllers and routes

We are now in the section where we will be writing controllers that process HTTP requests from the client application. We are also going to write routes that redirect an HTTP request to its respective controller.

These will be two simple controllers, but we will refactor them in the next chapter, *Chapter 6, Diving into CQRS*.

TourPackagesController

We will be creating the controller for `TourPackages` that will handle and process any requests that will retrieve and update `TourPackages` table in the database:

1. Now go back to the `Travel.WebApi` project and create `TourPackagesController.cs` inside of the `Controllers` directory and write the following code:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Threading.Tasks;
using Travel.Data.Contexts;
using Travel.Domain.Entities;
```

The code imports the preceding namespaces, which are needed in this controller.

2. Next, write the controller class:

```
namespace Travel.WebApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class TourPackagesController : ControllerBase
    {
    }
}
```

We will use the `ApiController` attribute, which gives the decorated class some API behaviors such as HTTP API responses, followed by another attribute, `Route` – the routes that redirects HTTP requests.

The `TourPackagesController` class is derived from `ControllerBase`, a base class for an MVC controller, but without the view support.

3. Next, write the following code inside `TourPackagesController`:

```
private readonly TravelDbContext _context;

public TourPackagesController(TravelDbContext
    context)
{
    _context = context;
}
```

We inject `TravelDbContext` in the constructor of `TourPackagesController`, which is then assigned to a backing field `_context`. Now let's move on to our first public method in the controller.

4. Next, we need to write the `Get` method under the constructor method:

```
[HttpGet]
public IActionResult Get()
{
    return Ok(_context.TourPackages);
}
```

The `HttpGet` attribute identifies an action that can support an `HttpGet` method. Also, the `Get` method returns a 200 status code along with a collection of `TourPackage`. This method gets triggered when the request's endpoint is `api/tourpackages` while using the `GET` HTTP verb request.

Now let's move on to the next public method.

5. Next, we need to write the `Create` method under the `Get` method:

```
[HttpPost]
public async Task<IActionResult> Create([FromBody]
    TourPackage tourPackage)
{
    await _context.TourPackages.AddAsync
```

```

        (tourPackage) ;
        await _context.SaveChangesAsync() ;

        return Ok(tourPackage) ;
    }

```

The `HttpPost` attribute on top of the `Create` method identifies an action that can support an `HttpPost` method. The post method takes a `TourPackage` object decorated with `FromBody`, specifying that a parameter should be bound using a request body.

The `Create` method returns a 200 status code along with newly created `TourPackage`. This method gets triggered when the request's endpoint is `api/tourpackages` while using the `POST` HTTP verb request.

You will also notice here `Task` of type `ActionResult`. `ActionResult` is an interface that defines a contract representing the result of an action method, while `Task` is used for asynchronous operations.

Now let's move on to the next public method of `TourPackagesController`.

6. Next, we write the `Delete` method under the `Create` method:

```

[HttpDelete("{id}")]
public async Task<ActionResult>
Delete([FromRoute] int id)
{
    var tourPackage = await
        _context.TourPackages.SingleOrDefaultAsync
            (tp => tp.Id == id);

    if (tourPackage == null)
    {
        return NotFound();
    }

    _context.TourPackages.Remove(tourPackage);
    await _context.SaveChangesAsync();

    return Ok(tourPackage);
}

```

The `HttpDelete` attribute on top of the `Delete` method identifies an action that can support an `HttpGet` method. This method gets triggered when the request's endpoint is `api/tourpackages/{id}` while using the `DELETE` HTTP verb request.

You will notice in the `HttpDelete` attribute of the method that there's an `"{id}"` string. `SingleOrDefaultAsync` uses the `id` parameter, matches it with an existing object, and then returns that object to be removed from the table. `DbContext` must call `SaveChangesAsync()` to finalize the deletion of the specified `TourPackage` object.

And finally, controller includes deleted `TourPackage` in the `200 - OK` response.

Now let's move on to the method.

7. We need to write the `Update` method under the `Delete` method:

```
[HttpPut("{id}")]
public async Task<IActionResult>
    Update([FromRoute] int id,
          [FromBody] TourPackage tourPackage)
{
    _context.Update(tourPackage);

    await _context.SaveChangesAsync();

    return Ok(tourPackage);
}
```

The preceding code is a method that updates a `TourPackage` object.

And that's it for `TourPackagesController`. We can now move on to `TourListsController`.

TourListsController

Now create `TourListsController` inside the `Controllers` directory. `TourListsController` has the same structure as `TourPackagesController`.

Here is the summary of the code for `TourListsController`. Go check out the code inside `Chapter 5 | Finish` directory of the application's GitHub repo and write it in your file:

```
using Microsoft.AspNetCore.Mvc;
...
using Travel.Domain.Entities;

namespace Travel.WebApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class TourListsController : ControllerBase
    {
        private readonly TravelDbContext _context;
        public TourListsController(TravelDbContext context) {...}

        [HttpGet]
        public IActionResult Get() {...}

        [HttpPost]
        public async Task<IActionResult> Create([FromBody]
            TourList tourList) {...}

        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete([FromRoute]
            int id) {...}

        [HttpPut("{id}")]
        public async Task<IActionResult> Update([FromRoute]
            int id, [FromBody] TourList tourList) {...}
    }
}
```

As you can see, the code is also a class that is derived from `ControllerBase` and has `Get`, `Create`, `Delete`, and `Update` public methods.

Now we've finished writing the controllers for the application, let's run it to see our work in action, by running the following command still inside the `WebApi` project:

```
dotnet run
```

The preceding command will start or run the application. While the application is running, we can move on to the next section, *Testing controllers with Swagger UI*, to see the controllers in action.

Testing controllers with Swagger UI

We've built controllers and run the application, but we haven't tested it yet, right? In this section, we will send **GET**, **POST**, **PUT**, and **DELETE** requests to `TourList` controller.

First of all, make sure that you are running the application by going to the `Web Api` project and running this command:

```
dotnet run
```

Now check your Swagger UI by going to the link `https://localhost:5001/swagger/index.html` while the application is running.

You will see the **TourLists** and **TourPackages** endpoints. The Swagger UI is showing you the documentation of the HTTP methods for each controller. There are **GET**, **POST**, **DELETE**, and **PUT** methods ready to be tried out:

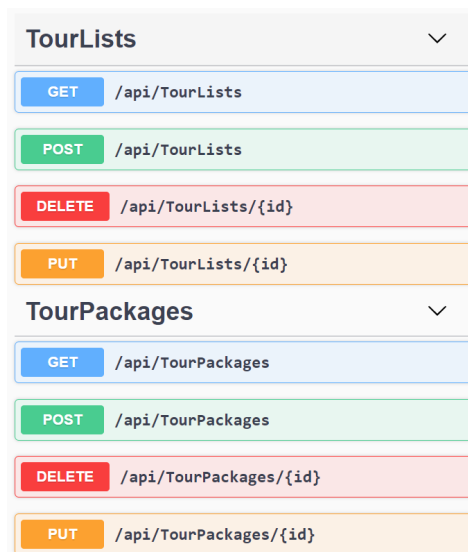


Figure 5.3 – Swagger UI

The following screenshot shows the schemas of the application, which are also generated for you by Swagger UI:

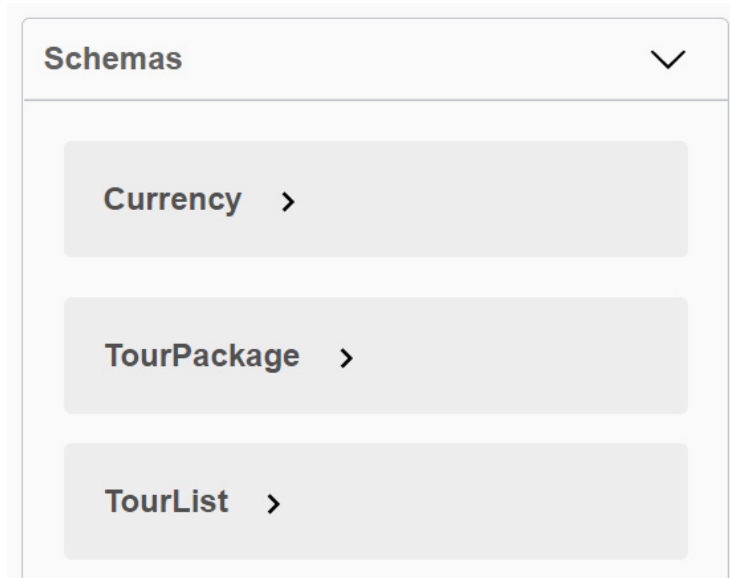


Figure 5.4 – Schemas in Swagger UI

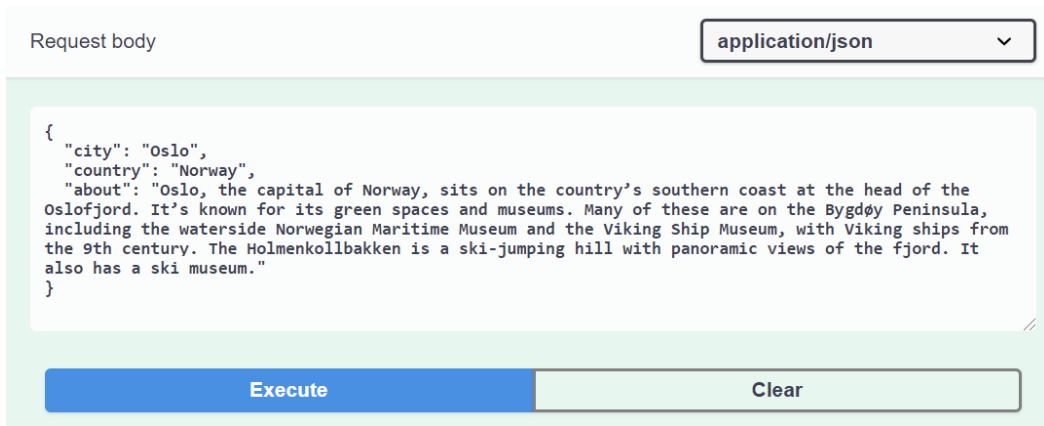
Now let's try the **GET** method of the `TourLists` controller. It will return an empty collection or array since we haven't added any data there yet.

Now go to **POST** and click the **Try it out** button. Before clicking the **Execute** button, replace the request body with the following code:

```
{
  "city": "Oslo",
  "country": "Norway",
  "about": "Oslo, the capital of Norway, sits on the country's
southern coast at the head of the Oslofjord. It's known
for its green spaces and museums. Many of these are on the
Bygdøy Peninsula, including the waterside Norwegian Maritime
Museum and the Viking Ship Museum, with Viking ships from the
9th century. The Holmenkollbakken is a ski-jumping hill with
panoramic views of the fjord. It also has a ski museum."
}
```

You can copy the preceding request body, which we will use to send a **POST** request to the server from the `Commands.txt` file that you can find inside the `Chapter 5` folder of the application's GitHub repo.

We will only include the `city` property, the `country` property, and the `about` property in the text area for the request body, like so:



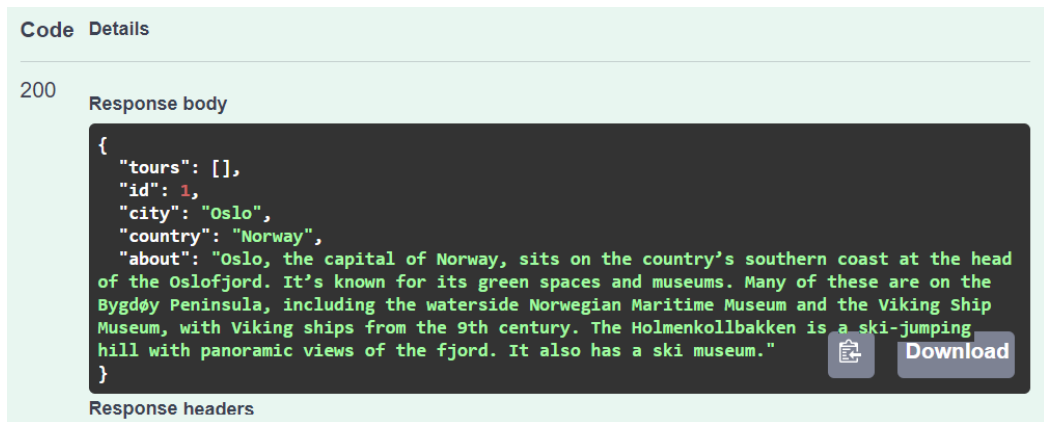
Request body application/json

```
{
  "city": "Oslo",
  "country": "Norway",
  "about": "Oslo, the capital of Norway, sits on the country's southern coast at the head of the
Oslofjord. It's known for its green spaces and museums. Many of these are on the Bygdøy Peninsula,
including the waterside Norwegian Maritime Museum and the Viking Ship Museum, with Viking ships from
the 9th century. The Holmenkollbakken is a ski-jumping hill with panoramic views of the fjord. It
also has a ski museum."
}
```

Execute **Clear**

Figure 5.5 – Request body for the POST method

After replacing the **POST** part's default request body, like you can see in *Figure 5.5*, click the **Execute** button:



Code Details

200 Response body

```
{
  "tours": [],
  "id": 1,
  "city": "Oslo",
  "country": "Norway",
  "about": "Oslo, the capital of Norway, sits on the country's southern coast at the head
of the Oslofjord. It's known for its green spaces and museums. Many of these are on the
Bygdøy Peninsula, including the waterside Norwegian Maritime Museum and the Viking Ship
Museum, with Viking ships from the 9th century. The Holmenkollbakken is a ski-jumping
hill with panoramic views of the fjord. It also has a ski museum."
}
```

Response headers **Download**

Figure 5.6 – Response body from the TourLists controller

You can see in *Figure 5.6* the response body has the properties we posted and an `id` of 1.

If you want to update the entry for Oslo City or delete it, you can go to Swagger UI and make the request there. However, since we'll also be using this in the application, I will leave it there as it is.

You can check out the whole finished solution of this chapter in the GitHub repository of this book.

Now we are done with this section, let's have a recap of what you have learned.

Summary

To summarize this chapter, we have covered that entities represent domain models in an application, and we write entities in the domain layer of the application.

We've also covered EF Core, which helps developers persist and query data from the database without using SQL queries through an O/RM.

We've learned that `DbContext` of Entity Framework is an interface of the database and has `DbSet` representing a table in the database.

And lastly, we've written two controllers with `DbContext` and tested them by sending HTTP requests through Swagger UI.

You now have the skills to handle any HTTP requests sent by any client-side applications and process and save them in any type of relational database using EF Core.

In the next chapter, we will learn how to write maintainable and testable code using **CQRS**, or **Command and Query Responsibility Segregation**. So, see you later.